

Step 1

```
In [ ]: import requests
        from bs4 import BeautifulSoup
        import pandas as pd

        # 1) 目标 URL
        base_url = "https://oig.hhs.gov"
        url = "https://oig.hhs.gov/fraud/enforcement/"

        # 2) 发出请求并解析
        response = requests.get(url)
        soup = BeautifulSoup(response.text, "lxml")

        # 3) 找到所有 enforcement action
        cards = soup.find_all("li", class_="usa-card")

        print(f"在第一页抓到 {len(cards)} 个 enforcement actions")

        # 4) 提取信息
        data = []

        for card in cards:
            # title + link
            h2_tag = card.find("h2")
            a_tag = h2_tag.find("a")
            title = a_tag.text.strip()
            link = base_url + a_tag["href"]

            # date
            span_tag = card.find("span")
            date = span_tag.text.strip()

            # category (取第一个 tag 作为分类)
            cat_tags = card.find_all("li", class_="usa-tag")
            category = cat_tags[0].text.strip() if cat_tags else None

            data.append({
                "title": title,
                "date": date,
                "category": category,
                "link": link
            })

        # 5) 变成 dataframe
        df = pd.DataFrame(data)

        # 6) 显示前几行
        print(df.head())

        # save_path = r"C:\Users\29947\Desktop\py2\step1_enforcement_page1.csv"
        # df.to_csv(save_path, index=False, encoding="utf-8-sig")

        # print("CSV 已保存到: ", save_path)
```

在第一页抓到 20 个 enforcement actions

	title	date \
0	Houston Transplant Doctor Indicted For Making ...	February 5, 2026
1	MultiCare Health System to Pay Millions to Set...	February 4, 2026
2	Brooklyn Banker Pleads Guilty to Laundering Pr...	February 3, 2026
3	Delafield Man Sentenced to 18 Months' Imprison...	February 3, 2026
4	Former NFL Player Convicted for \$197M Medicare...	February 3, 2026

	category \
0	Criminal and Civil Actions
1	Criminal and Civil Actions
2	COVID-19
3	Criminal and Civil Actions
4	Criminal and Civil Actions

	link
0	https://oig.hhs.gov/fraud/enforcement/houston-...
1	https://oig.hhs.gov/fraud/enforcement/multicar...
2	https://oig.hhs.gov/fraud/enforcement/brooklyn...
3	https://oig.hhs.gov/fraud/enforcement/delafiel...
4	https://oig.hhs.gov/fraud/enforcement/former-n...

STEP 2: a

Pseudo-code:

A: Define a function that takes a starting year and month as inputs

B: If the year is earlier than 2013, print a message and exit the function

C: Initialize an empty list to store enforcement actions

D: Start from page 1 and loop through pages using a while loop

E: For each page:

1. Construct the URL with the page number
2. Request the webpage and parse the HTML
3. Extract title, date, category, and link for each enforcement action
4. Append results to the list
5. Check the oldest date on the page
6. If the date is earlier than the starting date, stop looping
7. Pause for 1 second before requesting the next page

F: Convert the collected data into a dataframe

G: Save the dataframe as a CSV file and return it

```

In [11]: # STEP 2: b
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
from datetime import datetime

def scrape_enforcement_actions(start_year, start_month, RUN_SCRAPER=False):
    """
    Scrape HHS OIG enforcement actions from a given start year/month to today.
    Step 2b version: does NOT save CSV yet.
    """

    # ---- 安全开关 ----
    if not RUN_SCRAPER:
        print("RUN_SCRAPER is False. Skipping web scraping.")
        return None

    # ---- 年份检查 ----
    if start_year < 2013:
        print("Error: Enforcement Actions are only available from 2013 onward.")
        return None

    base_url = "https://oig.hhs.gov"
    data = []

    start_date = datetime(start_year, start_month, 1)

    page = 1
    keep_scraping = True

    while keep_scraping:
        print(f"Scraping page {page}...")

        url = f"https://oig.hhs.gov/fraud/enforcement/?page={page}"
        response = requests.get(url)
        soup = BeautifulSoup(response.text, "lxml")

        cards = soup.find_all("li", class_="usa-card")

        if len(cards) == 0:
            break

        for card in cards:
            a_tag = card.find("h2").find("a")
            title = a_tag.text.strip()
            link = base_url + a_tag["href"]

            date_str = card.find("span").text.strip()
            date = datetime.strptime(date_str, "%B %d, %Y")

            categories = card.find_all("li", class_="usa-tag")
            category = categories[0].text.strip() if categories else None

            # ---- 停止条件 ----
            if date < start_date:
                keep_scraping = False
                break

```

```
        data.append({
            "title": title,
            "date": date,
            "category": category,
            "link": link
        })

    page += 1
    time.sleep(1)

    df = pd.DataFrame(data)
    print(f"Scraped {len(df)} enforcement actions.")
    return df

df_2024 = scrape_enforcement_actions(2024, 1, RUN_SCRAPER=True)

df_2024.sort_values("date").head(1)
```

Scraping page 1...
Scraping page 2...
Scraping page 3...
Scraping page 4...
Scraping page 5...
Scraping page 6...
Scraping page 7...
Scraping page 8...
Scraping page 9...
Scraping page 10...
Scraping page 11...
Scraping page 12...
Scraping page 13...
Scraping page 14...
Scraping page 15...
Scraping page 16...
Scraping page 17...
Scraping page 18...
Scraping page 19...
Scraping page 20...
Scraping page 21...
Scraping page 22...
Scraping page 23...
Scraping page 24...
Scraping page 25...
Scraping page 26...
Scraping page 27...
Scraping page 28...
Scraping page 29...
Scraping page 30...
Scraping page 31...
Scraping page 32...
Scraping page 33...
Scraping page 34...
Scraping page 35...
Scraping page 36...
Scraping page 37...
Scraping page 38...
Scraping page 39...
Scraping page 40...
Scraping page 41...
Scraping page 42...
Scraping page 43...
Scraping page 44...
Scraping page 45...
Scraping page 46...
Scraping page 47...
Scraping page 48...
Scraping page 49...
Scraping page 50...
Scraping page 51...
Scraping page 52...
Scraping page 53...
Scraping page 54...
Scraping page 55...
Scraping page 56...
Scraping page 57...
Scraping page 58...
Scraping page 59...
Scraping page 60...

```
Scraping page 61...
Scraping page 62...
Scraping page 63...
Scraping page 64...
Scraping page 65...
Scraping page 66...
Scraping page 67...
Scraping page 68...
Scraping page 69...
Scraping page 70...
Scraping page 71...
Scraping page 72...
Scraping page 73...
Scraping page 74...
Scraping page 75...
Scraping page 76...
Scraping page 77...
Scraping page 78...
Scraping page 79...
Scraping page 80...
Scraping page 81...
Scraping page 82...
Scraping page 83...
Scraping page 84...
Scraping page 85...
Scraping page 86...
Scraping page 87...
Scraping page 88...
Scraping page 89...
Scraping page 90...
Scraped 1787 enforcement actions.
```

Out[11]:

	title	date	category	link
1786	Former Nurse Aide Indicted In Death Of Clarksv...	2024-01-03	State Enforcement Agencies	https://oig.hhs.gov/fraud/enforcement/former-n...

Using my scraping function, I collected enforcement actions beginning in January 2024. In total, the scraper returned 1,787 observations. The earliest enforcement action in this dataset occurred on January 3, 2024 and is titled "Former Nurse Aide Indicted In Death Of Clarksville..."

```
In [12]: # STEP 2 C

def scrape_enforcement_actions(start_year, start_month, RUN_SCRAPER=False):
    """
    Scrape HHS OIG enforcement actions from a given start year/month to today.
    Step 2c version: saves results to CSV.
    """

    # ---- 安全开关 ----
    if not RUN_SCRAPER:
        print("RUN_SCRAPER is False. Skipping web scraping.")
```

```

    return None

# ---- 年份检查 ----
if start_year < 2013:
    print("Error: Enforcement Actions are only available from 2013 onward.")
    return None

base_url = "https://oig.hhs.gov"
data = []

start_date = datetime(start_year, start_month, 1)

page = 1
keep_scraping = True

while keep_scraping:
    print(f"Scraping page {page}...")

    url = f"https://oig.hhs.gov/fraud/enforcement/?page={page}"
    response = requests.get(url)
    soup = BeautifulSoup(response.text, "lxml")

    cards = soup.find_all("li", class_="usa-card")

    if len(cards) == 0:
        break

    for card in cards:
        a_tag = card.find("h2").find("a")
        title = a_tag.text.strip()
        link = base_url + a_tag["href"]

        date_str = card.find("span").text.strip()
        date = datetime.strptime(date_str, "%B %d, %Y")

        categories = card.find_all("li", class_="usa-tag")
        category = categories[0].text.strip() if categories else None

        # ---- 停止条件 ----
        if date < start_date:
            keep_scraping = False
            break

        data.append({
            "title": title,
            "date": date,
            "category": category,
            "link": link
        })

    page += 1
    time.sleep(1)

df = pd.DataFrame(data)

filename = f"enforcement_actions_{start_year}_{start_month}.csv"
df.to_csv(filename, index=False)

print(f"Saved {len(df)} rows to {filename}")
return df

```

```
In [13]: df_2022 = scrape_enforcement_actions(2022, 1, RUN_SCRAPER=True)
```


Scraping page 1...
Scraping page 2...
Scraping page 3...
Scraping page 4...
Scraping page 5...
Scraping page 6...
Scraping page 7...
Scraping page 8...
Scraping page 9...
Scraping page 10...
Scraping page 11...
Scraping page 12...
Scraping page 13...
Scraping page 14...
Scraping page 15...
Scraping page 16...
Scraping page 17...
Scraping page 18...
Scraping page 19...
Scraping page 20...
Scraping page 21...
Scraping page 22...
Scraping page 23...
Scraping page 24...
Scraping page 25...
Scraping page 26...
Scraping page 27...
Scraping page 28...
Scraping page 29...
Scraping page 30...
Scraping page 31...
Scraping page 32...
Scraping page 33...
Scraping page 34...
Scraping page 35...
Scraping page 36...
Scraping page 37...
Scraping page 38...
Scraping page 39...
Scraping page 40...
Scraping page 41...
Scraping page 42...
Scraping page 43...
Scraping page 44...
Scraping page 45...
Scraping page 46...
Scraping page 47...
Scraping page 48...
Scraping page 49...
Scraping page 50...
Scraping page 51...
Scraping page 52...
Scraping page 53...
Scraping page 54...
Scraping page 55...
Scraping page 56...
Scraping page 57...
Scraping page 58...
Scraping page 59...
Scraping page 60...

Scraping page 61...
Scraping page 62...
Scraping page 63...
Scraping page 64...
Scraping page 65...
Scraping page 66...
Scraping page 67...
Scraping page 68...
Scraping page 69...
Scraping page 70...
Scraping page 71...
Scraping page 72...
Scraping page 73...
Scraping page 74...
Scraping page 75...
Scraping page 76...
Scraping page 77...
Scraping page 78...
Scraping page 79...
Scraping page 80...
Scraping page 81...
Scraping page 82...
Scraping page 83...
Scraping page 84...
Scraping page 85...
Scraping page 86...
Scraping page 87...
Scraping page 88...
Scraping page 89...
Scraping page 90...
Scraping page 91...
Scraping page 92...
Scraping page 93...
Scraping page 94...
Scraping page 95...
Scraping page 96...
Scraping page 97...
Scraping page 98...
Scraping page 99...
Scraping page 100...
Scraping page 101...
Scraping page 102...
Scraping page 103...
Scraping page 104...
Scraping page 105...
Scraping page 106...
Scraping page 107...
Scraping page 108...
Scraping page 109...
Scraping page 110...
Scraping page 111...
Scraping page 112...
Scraping page 113...
Scraping page 114...
Scraping page 115...
Scraping page 116...
Scraping page 117...
Scraping page 118...
Scraping page 119...
Scraping page 120...

Scraping page 121...
Scraping page 122...
Scraping page 123...
Scraping page 124...
Scraping page 125...
Scraping page 126...
Scraping page 127...
Scraping page 128...
Scraping page 129...
Scraping page 130...
Scraping page 131...
Scraping page 132...
Scraping page 133...
Scraping page 134...
Scraping page 135...
Scraping page 136...
Scraping page 137...
Scraping page 138...
Scraping page 139...
Scraping page 140...
Scraping page 141...
Scraping page 142...
Scraping page 143...
Scraping page 144...
Scraping page 145...
Scraping page 146...
Scraping page 147...
Scraping page 148...
Scraping page 149...
Scraping page 150...
Scraping page 151...
Scraping page 152...
Scraping page 153...
Scraping page 154...
Scraping page 155...
Scraping page 156...
Scraping page 157...
Scraping page 158...
Scraping page 159...
Scraping page 160...
Scraping page 161...
Scraping page 162...
Scraping page 163...
Scraping page 164...
Scraping page 165...
Scraping page 166...
Scraping page 167...
Scraping page 168...
Scraping page 169...
Saved 3377 rows to enforcement_actions_2022_1.csv

```
In [15]: import pandas as pd
import altair as alt

# 重新读 CSV
df = pd.read_csv("enforcement_actions_2022_1.csv")

# 把 date 转成 datetime (非常重要)
df["date"] = pd.to_datetime(df["date"])
```

```
# 创建 year_month 列 (关键一步)
df["year_month"] = df["date"].dt.to_period("M").dt.to_timestamp()

df.head()
```

Out[15]:

	title	date	category	link	year
0	Houston Transplant Doctor Indicted For Making ...	2026-02-05	Criminal and Civil Actions	https://oig.hhs.gov/fraud/enforcement/houston-...	20
1	MultiCare Health System to Pay Millions to Set...	2026-02-04	Criminal and Civil Actions	https://oig.hhs.gov/fraud/enforcement/multicar...	20
2	Brooklyn Banker Pleads Guilty to Laundering Pr...	2026-02-03	COVID-19	https://oig.hhs.gov/fraud/enforcement/brooklyn...	20
3	Delafield Man Sentenced to 18 Months' Imprison...	2026-02-03	Criminal and Civil Actions	https://oig.hhs.gov/fraud/enforcement/delafiel...	20
4	Former NFL Player Convicted for \$197M Medicare...	2026-02-03	Criminal and Civil Actions	https://oig.hhs.gov/fraud/enforcement/former-n...	20



In [17]:

```
# Step 3
# 1

monthly_total = (
    df.groupby("year_month")
      .size()
      .reset_index(name="count")
)

chart_total = (
    alt.Chart(monthly_total)
      .mark_line(point=True)
      .encode(
        x=alt.X("year_month:T", title="Month"),
        y=alt.Y("count:Q", title="Number of Enforcement Actions"),
        tooltip=["year_month:T", "count:Q"]
      )
      .properties(
        title="Monthly Enforcement Actions (Since Jan 2022)",
        width=650,

```

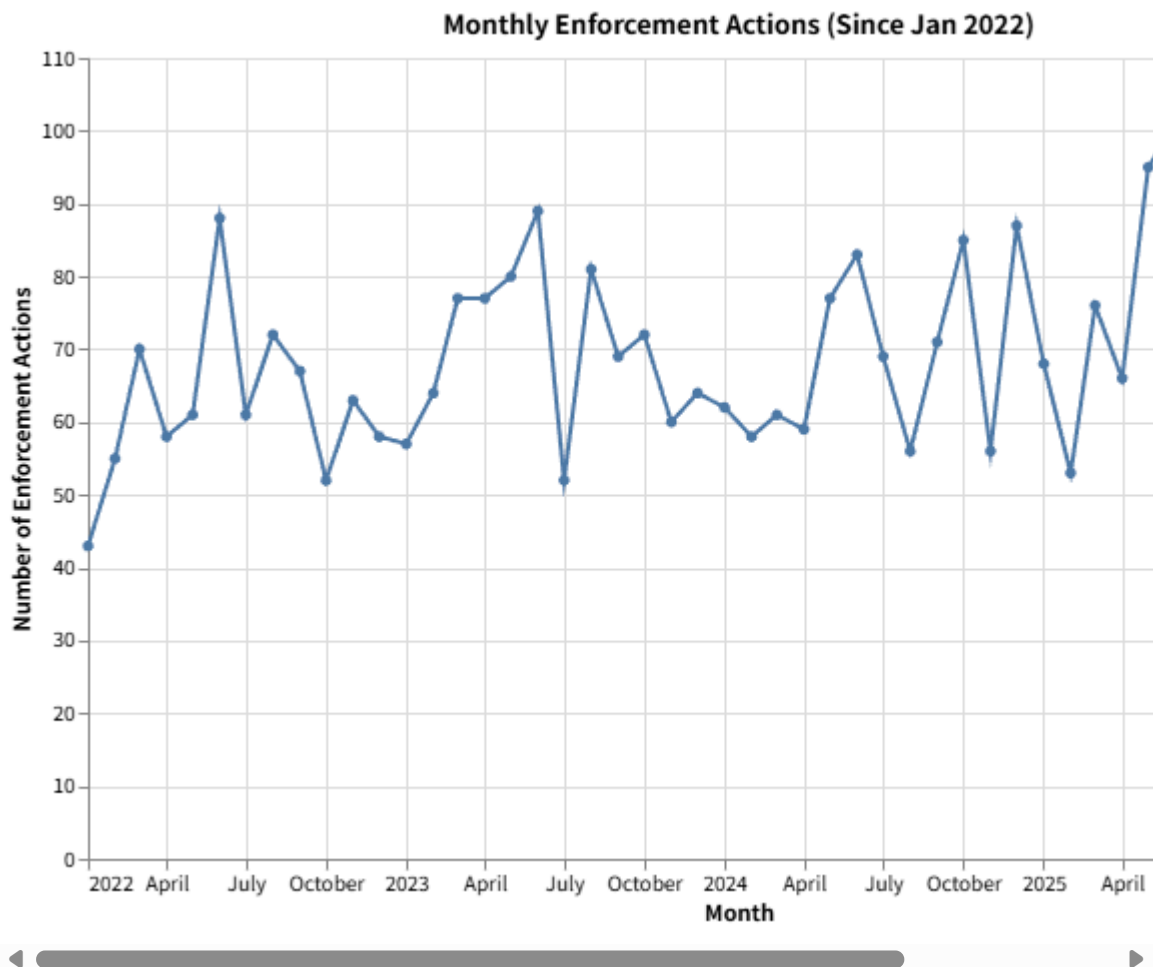
```

        height=400
    )
)

chart_total

```

Out[17]:



In [20]: # 2

```

focus_categories = [
    "Criminal and Civil Actions",
    "State Enforcement Agencies"
]

df_two = df[df["category"].isin(focus_categories)]
df_two["category"].value_counts()

# Combine

monthly_two = (
    df_two
    .groupby(["year_month", "category"])
    .size()
    .reset_index(name="count")
)

monthly_two.head()

chart_two = (
    alt.Chart(monthly_two)

```

```

.mark_line(point=True)
.encode(
    x=alt.X("year_month:T", title="Month"),
    y=alt.Y("count:Q", title="Number of Enforcement Actions"),
    color=alt.Color("category:N", title="Category"),
    tooltip=["year_month:T", "category:N", "count:Q"]
)
.properties(
    title="Monthly Enforcement Actions by Category (Since Jan 2022)",
    width=650,
    height=400
)
)

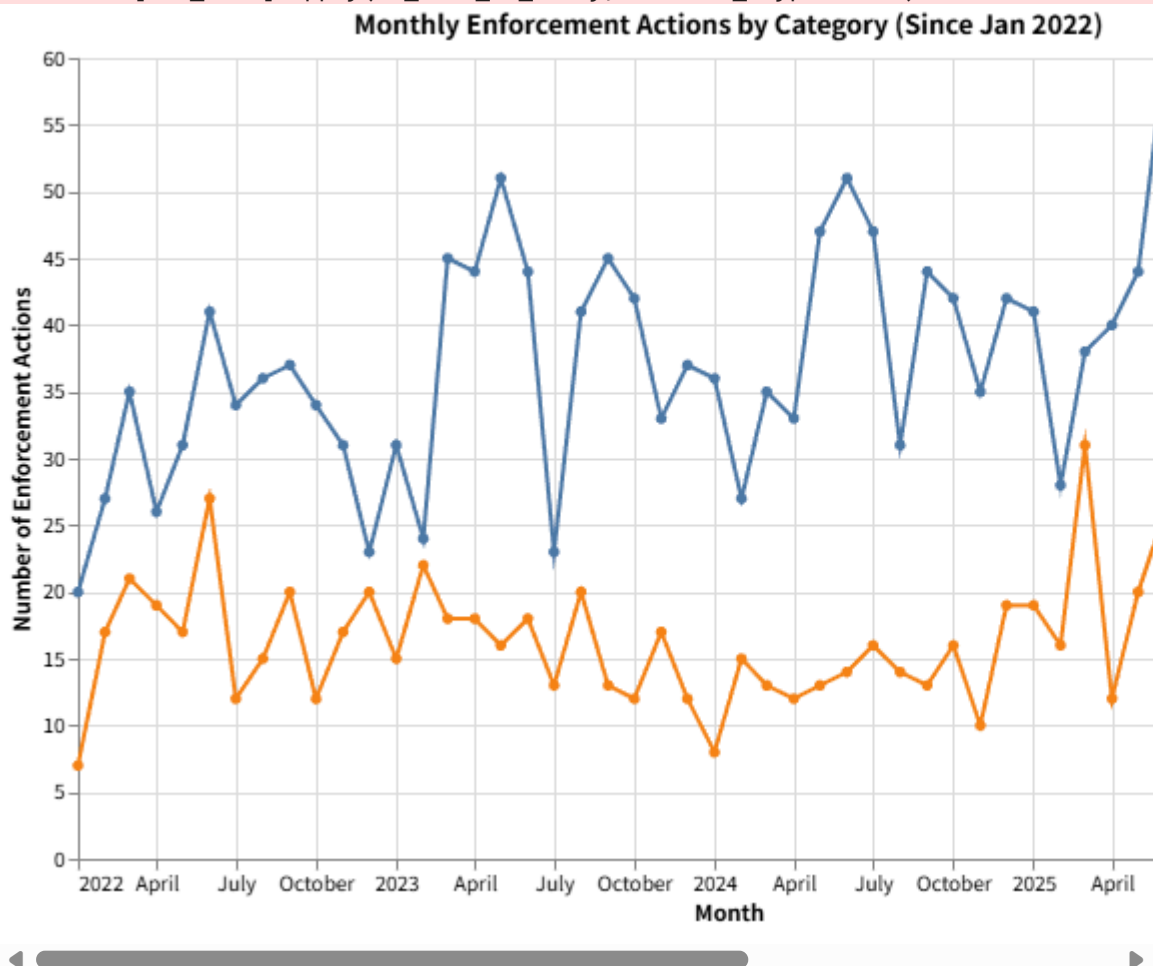
chart_two

```

e:\Anaconda\Lib\site-packages\altair\utils\core.py:395: FutureWarning: the convert_dtype parameter is deprecated and will be removed in a future version. Do ``series.apply(object).apply()`` instead if you want ``convert\_dtype=False``.

col = df[col_name].apply(to_list_if_array, convert_dtype=False)

Out[20]:



In [25]:

```

# Step 3

def classify_criminal_action(title):
    t = title.lower()

    if any(k in t for k in ["medicare", "medicaid", "health care", "healthcare"]):
        return "Health Care Fraud"

    elif any(k in t for k in ["bank", "wire fraud", "launder", "financial"]):
        return "Financial Fraud"

```

```

elif any(k in t for k in ["drug", "opioid", "controlled substance"]):
    return "Drug Enforcement"

elif any(k in t for k in ["bribery", "kickback", "corruption"]):
    return "Bribery / Corruption"

else:
    return "Other"

df_criminal = df[df["category"] == "Criminal and Civil Actions"].copy()

df_criminal["subtype"] = df_criminal["title"].apply(classify_criminal_action)

df_criminal["subtype"].value_counts()

# Combine
monthly_criminal = (
    df_criminal
    .groupby(["year_month", "subtype"])
    .size()
    .reset_index(name="count")
)

monthly_criminal.head()

chart_criminal = (
    alt.Chart(monthly_criminal)
    .mark_line(point=True)
    .encode(
        x=alt.X("year_month:T", title="Month"),
        y=alt.Y("count:Q", title="Number of Actions"),
        color=alt.Color("subtype:N", title="Criminal Action Type"),
        tooltip=["year_month:T", "subtype:N", "count:Q"]
    )
    .properties(
        title="Criminal and Civil Actions by Topic (Since Jan 2022)",
        width=650,
        height=400
    )
)

chart_criminal

```

```

e:\Anaconda\Lib\site-packages\altair\utils\core.py:395: FutureWarning: the convert_dtype parameter is deprecated and will be removed in a future version. Do ``series.astype(object).apply()`` instead if you want ``convert_dtype=False``.
col = df[col_name].apply(to_list_if_array, convert_dtype=False)

```

Out[25]:

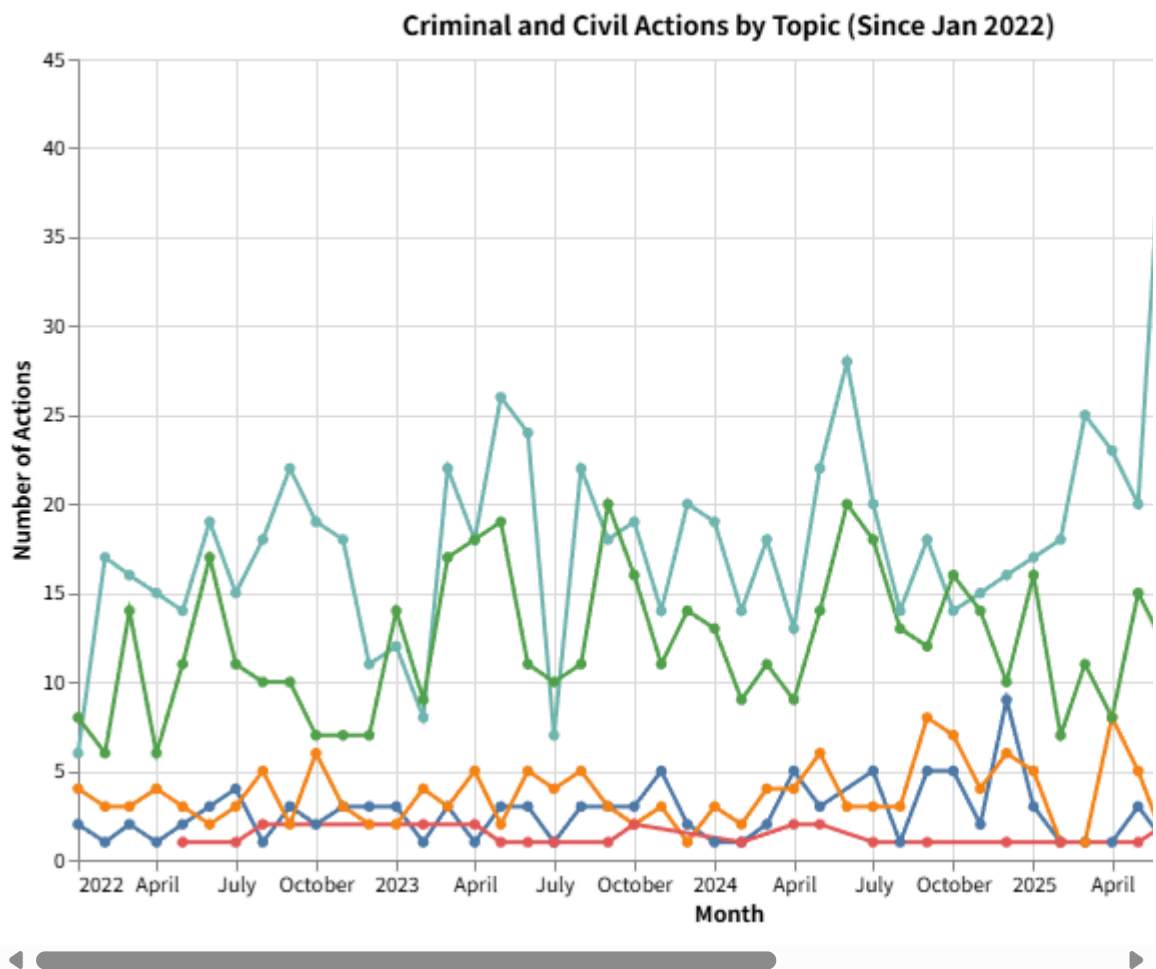


Figure 3 further disaggregates Criminal and Civil Actions into five topic areas—Health Care Fraud, Financial Fraud, Drug Enforcement, Bribery/Corruption, and Other—based on keyword classification of enforcement action titles. Monthly counts are plotted to illustrate trends across these subcategories over time.